

Föreläsning 21

Ändrad 15 Oct 12:55

This is not The End!

Nu är det så här förstår ni, att det viktigaste är genomgången. Men visst finns det mer. Mycket spännande saker att välja och vraka emellan.

Jag har skrivit upp på gula lappar smått och gott som återstår och rangordnat dem i "Nödvändigt", "Önskvärt" och "Kan offras".

Har någon åsikter om rangordningen? Hör av er snabbt!

[Snabla mig ett snabelbrev!](#)

Nödvändigt

- Separat kompilering. -- en fil med enbart funktionsdeklarationer kan kompileras. Därefter kan funktionerna användas i andra program utan omkompilering. Nödvändigt vid större projekt med mycket kod och/eller många medarbetare.
- `static`, `extern` -- globala deklarationer, räckvidd -- variabler kan deklareras så att de är åtkomliga i flera funktioner samtidigt.
- Historik, kontext.
- `while(i--)` -- bryter för noll == false
- 2- och flerdimensionella fält
- `break`, `exit`, `continue`
- Mer om `typedef`
- Kommaoperatorn

Önskvärt

- Exempel: sortering efter nyckel
- Exempel: Quick sort eller Merge sort
- Rekursion
- Bitfält -- variabler packade som sillar
- `union` -- schizofrena variabler
- Makro med parameter -- nästan som funktion
- `scanf()` -- matchar tecken, ger returvärde
- `sscanf()`, `sprintf()` -- läs från/till sträng
- Bitoperationer `&`, `|`, `<<`, `>>`, `~`, `^` -- maska fram den intressanta biten
- handtag (= pekare på pekare) och ännu högre ordningens pekare
- `malloc` och `free` -- dynamisk minnesallokering

Kan offras

- Exempel: stack
- `register` -- lagra gärna i register
- `argv, argc` -- parametrar till `main()`
- Mer flexibel filhantering -- `fseek()` -- läs *och* skriv var du vill
- Kompileringsdirektiv -- `ifdef`
- Rekursiva strukturer -- länkade listor -- egen kurs!
- Funktioner som parametrar -- praktiskt men jobbig syntax
- Variabelt antal argument -- i likhet med `printf()`
- Fler strängfunktioner -- det kan ni väl kolla upp själva?
- Andra funktionsbibliotek
- Parallella processer

Generera klassens användarnamn

Följande exempel är mest ett axplock gamla kunskaper. Men dessutom är det ett litet större exempel. Och dessutom får ni se ett par nya saker, bland annat några nya möjligheter hos `scanf()`.

Programmet läser in en fil med studenter:

```
1 Karlsson Olof, 630807-6635, FFG,TIDAY
2 Olsson Karl, 630809-6635, FFG, TIELY
3 Ottosson Daniel, 630810-6635, FFG,TIDEY
4 Hansson Otto, 630811-6635, FFG,TIELY
5 Danielsson Hans, 630812-6635, FFG,TIDAY
```

Programmet beräknar bland annat studentens användarnamn, utifrån studentuppgifterna. Därefter skrivs studenterna ut på `stdout`, i ett nytt format:

```
1 Namn: Karlsson Olof (di97okn)
2 Personnummer: 630807-6635
3 Program: TIDAY
4
5 Namn: Olsson Karl (ei97kon)
6 Personnummer: 630809-6635
7 Program: TIELY
8
9 Namn: Ottosson Daniel (??97don)
10 Personnummer: 630810-6635
11 Program: TIDEY
12
13 Namn: Hansson Otto (ei97ohn)
14 Personnummer: 630811-6635
15 Program: TIELY
16
17 Namn: Danielsson Hans (di97hdn)
18 Personnummer: 630812-6635
19 Program: TIDAY
20
```

I programmet nedan definieras två `struct`ar:

- `student` håller reda på allt som är värt att veta om en student.

Ett värde av typen `student` kräver 60 byte på mitt system
 (= `MAXNAME + 8 + 6 + sizeof(unsigned long) + sizeof(short)`)

- `class` håller reda på varje student i en hel klass. Fältet `s`

lagrar studenterna i en array, och fältet `size` håller reda på arrayens storlek, dvs antalet studenter i klassen.

Ett värde av typen `class` kräver 6002 byte (= `sizeof(student) + sizeof(unsigned short)`).

Här följer hela programmet:

```
1  #include <stdio.h>
2  #include <string.h>
3  #include <ctype.h>
4
5  /* Exempel 50. PT DING + EING 1997.
6
7      Läs in studentdata från fil.
8      Generera username enligt regel första-första-sista.
9      Skriv på stdout i delvis nytt format.
10 */
11
12 #define MAXLINE 100
13 #define MAXSTUDENTS 100
14 #define MAXNAME 40
15
16
17 typedef struct {
18     char name[MAXNAME];      /* Ex: "Tångring, Jan" */
19     char user[8];            /* Ex: "di97jtn" */
20     char program[6];         /* Ex: "TIDAY" */
21     unsigned long birthdate; /* 97-07-02 == 19970702 */
22     short control;          /* Sista fyra siffrorna */
23 } student;
24
25
26 typedef struct {
27     unsigned short size;
28     student s[MAXSTUDENTS];
29 } class;
30
31
32 void print_pnr(student s)
33 {
34     printf("%.6d-%.4d\n", s.birthdate % 1000000, s.control);
35 }
36
37
38 void print_student(student s)
39 {
40     printf("Namn: %s (%s)\n", s.name, s.user);
41     printf("Personnummer: ");
42     print_pnr(s);
43     printf("Program: %s\n\n", s.program);
44 }
45
46
47 void print_class(class c)
48 {
49     int i;
50
51     for (i=0; i < c.size; i++)
52         print_student(c.s[i]);
53 }
54
55
56 void make_username(student *s)
57 {
58     char first[MAXNAME], last[MAXNAME];
59
60     if (strcmp(s->program, "TIDAY") == 0)
61         strcpy(s->user, "di");
62     else if (strcmp(s->program, "TIELY") == 0)
63         strcpy(s->user, "ei");
64     else
65         strcpy(s->user, "??");
66
67     strcat(s->user, "97");
68
69     sscanf(s->name, "%s%s", last, first);
70
71     s->user[4] = tolower(first[0]);
72     s->user[5] = tolower(last[0]);
73     s->user[6] = tolower(last[strlen(last) - 1]);
```

```

74     s -> user[7] = '\0';
75 }
76
77
78 /* Läs en student från fil med rader i format:
79    "Hansson Jan, 630811-6635, FFG, TIDAY"
80 */
81 int fscanstudent(FILE *f, student *s)
82 {
83     int i;
84     i = fscanf(f, "%[^,],%d-%hd, FFG, %s ",
85               &(s -> name),
86               &(s -> birthdate),
87               &(s -> control),
88               &(s -> program));
89     s -> birthdate = 19000000 + s -> birthdate;
90
91     return i;
92 }
93
94
95 void read_class(char filename[], class *c)
96 {
97     FILE *f = fopen(filename, "r");
98     int i = 0;
99
100     while (fscanstudent(f, &(c -> s[i])) == 4) {
101         make_username(&(c -> s[i]));
102         i++;
103     }
104     c -> size = i;
105     fclose(f);
106 }
107
108
109 void main()
110 {
111     class c;
112
113     read_class("e50.input", &c);
114     print_class(c);
115 }

```

Ovanstående program innehåller sju funktionsdefinitioner:

`print_pnr()`, `print_student()`, `print_class()`,
`make_username()`, `fscanstudent()`, `read_class()`, samt
`main()`.

Huvudfunktionen `main()` anropar funktionerna
`read_class()` och `print_class()` som i sin tur anropar
programmets övriga funktioner.

Funktionerna förklarade

■ `print_class()` skriver ut samtliga studenter i fältet `c.s`.
Antalet ges av `c.size` -- detta är programmerarens egen
konvention.

Eftersom parametern `c` är en `class`, och en klass består
av 6002 byte, kopieras 6002 byte varje gång `print_class`
`()` anropas. Detta tar tid och plats. Det vore mer ekonomiskt
att ha parametern `c` som en pekare till `class`, som
exempelvis i funktionen `read_class()`.

■ `make_username()` anropar strängfunktionerna `strcmp()`,
`strcat()` och `strcpy()` från biblioteket `strings.h`.

Funktionerna beskrivs i kapitel 10.8 i [Bilting/Skansholm](#).

-- `strcmp()` avgör likhet mellan två strängar. Den kan
även användas för att avgöra vilken av strängarna som
kommer först i bokstavsordning.

-- `strcpy(s1, s2)` kopierar strängen `s2` till strängen `s1`.
Om du bara skulle skriva `s1 = s2` skulle endast adresser kopieras.
-- `strcat(s1, s2)` adderar strängen `s2` till slutet av `s1`.
Identisk med funktionen `append()` från [Gruppövning 3](#).

■ `fscanstudent()` läser in en rad som ser ut enligt kommentaren ovanför funktionen.

Här görs ett märkligt anrop av `fscanf` som läser in hela raden till fyra variabler. Om anropet till `fscanf()` lyckas, returneras värdet `4` till `i` = antalet inlästa variabler.

Formatsträngen beskriver hur `fscanf()` förväntar sig att raden ska se ut. Vid minsta avvikelse misslyckas inläsningen, och `fscanf()` returnerar med ett värde som är mindre än fyra.

Formatsträngen tecken för tecken:

■ `"%[^,]"` är formatsträng för inläsning till en sträng (precis som `"%s"`, fast med fler tillvalsmöjligheter). Datorn läser in tecken till strängen till dess att ett komma dyker upp. Tecknet `^` betyder INTE -- dvs strängen ska INTE innehålla kommatecken.

Inom hakparenteserna räknar man upp de tecken som ska sätta stopp för inläsningen. Denna gång endast tecknet `,`.

Ta bort `^` så vänder vi på steken, då ska den inlästa strängen *bara* innehålla tecknen inom hakparenteserna. Alla andra tecken gör att inläsningen till strängen avslutas.

Strängen hamnar i fältet `s -> name`.

■ `","` matchar kommatecknet efter namnet. Datorn konstaterar att tecknet ligger där, och hoppar över det -- det hamnar inte i någon variabel.

■ `"%d"` läser in en `int` till `s -> birthdate`

■ `"-"` matchar ett minus. Inget läses in.

■ `"%hd"` läser in en `short` till `s -> control`

■ `"," FFG, "` matchar dessa tecken (närmare bestämt matchar blanktecknen mer än varsitt blanktecken -- de matchar varsin godtyckligt lång följd av tomrum).

■ `"%s"` läser in en sträng till `s -> program`

■ `" "` matchar godtyckligt lång följd av tomrum (i detta fall endast nyradstecknet, är det tänkt).

■ `read_class()` anropar `fscanstudent()` tills den misslyckas -- vilket vi tolkar som att filen är slut.

För varje inläst värde till fältet `c -> s[i]`, anropar vi

funktionen `make_username()` som konstruerar ett lämpligt värde till fältet `s -> s[i].user`.

Avslutningsvis sätts fältet `c -> size` till antalet inlästa värden.

Programflöde

Allmänt gäller följande: satserna i funktionen `main()` utförs alltid. Övriga funktioner utförs endast om de anropas. Såhär läser datorn igenom programmet:

- Satserna i funktionen `main()` utförs, i tur och ordning
- Allteftersom andra funktioner anropas, utförs dessa. Eventuella parametrar kopieras från anropet till funktionen.
Den anropade funktionens satser utförs. Om nya funktioner anropas, utförs dessa, etc.
Eventuella returvärden returneras i en `return`-sats.

Här är ett "skelett" av programmet som visar programflödet. Jag har bara behållit funktionsanrop, villkorssatser, och loopsatser.

```

32 void print_pnr(student s)
33 { }
37
38 void print_student(student s)
39 { print_pnr(...) }
46
47 void print_class(class c)
48 { for (...)
49     print_student(...)
53 }
55
56 void make_username(student *s)
57 {
60     if (strcmp(...)
61         strcpy(...)
62     else if (strcmp(...)
63         strcpy(...)
65     else
66         strcpy(...)
67     strcat(...)
68     sscanf(...)
71     tolower(...)
72     tolower(...)
73     tolower(...)
75 }
77
81 int fscanstudent(FILE *f, student *s)
82 {
84     i = fscanf(...)
91     return i
92 }
94
95 void read_class(char filename[], class *c)
96 {
97     fopen(...)
100     while (fscanstudent(...) == 4)
101         make_username(...)
105     fclose(f)
106 }
108
109 void main()
110 {
113     read_class(...)
114     print_class(...)

```

115 }

Nästa lektion

